

Pose, Velocity and the Differential Drive

A Fifteen-Slide Summary

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 4, Lecture A (light)



Pose in the Plane

A robot on a flat floor has exactly three degrees of freedom

x and y give the position, in metres. θ gives the orientation, in radians. These three values are sufficient and necessary: two robots at the same (x, y) with different orientations are in different states.

$$\text{pose} = (x, y, \theta) \quad \theta \in (-\pi, \pi]$$

Symbol	Units	Read it as
x, y	metres	position of the robot centre in the world frame
θ	radians	orientation, measured anticlockwise from the x axis

Three values. Every topic in this lecture transmits some form of them.

Two Representations of Orientation

turtlesim provides θ directly.

```
$ ros2 topic echo /turtle1/pose
x: 5.544445
y: 5.544445
theta: 0.0
```

A single float. No conversion is required.

A **physical robot** provides a quaternion.

```
$ ros2 topic echo /odom
orientation:
  x: 0.0    y: 0.0
  z: 0.383  w: 0.924
```

Four values, always of unit norm. You compute the conversion.

Under the hood

A quaternion has no discontinuity at $\pm\pi$ and no gimbal lock. This is why robots that rotate in three dimensions use one. The cost is that θ must be computed: $\theta = \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2))$.

Velocity and the Twist Message

```
geometry_msgs/msg/Twist
```

```
Vector3 linear    x y z
```

```
Vector3 angular   x y z
```

A differential-drive robot responds to **two** of them:

- `linear.x` — forward speed v , m/s
- `angular.z` — turn rate ω , rad/s

The other four are valid to transmit and produce no motion.

v	ω	Path	Turtle draws
> 0	0	straight line	a line
0	$\neq 0$	spin in place	a dot
> 0	$\neq 0$	arc of radius $R = v/\omega$	a circle
> 0	changing	spiral	a spiral

Two values determine the shape of the path.

The Nonholonomic Constraint

```
$ ros2 topic echo /turtle1/pose --once      x: 5.544445  y: 5.544445
$ ros2 run ee414_course simple_nonholonomic publishes linear.y = 2.0
$ ros2 topic echo /turtle1/pose --once      x: 5.544445  y: 5.544445
```

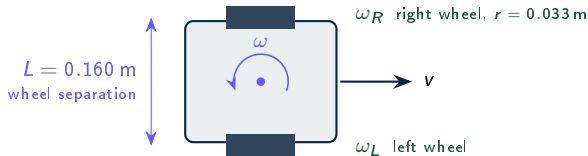
No error is raised, and nothing moves

The *message* is valid: the field exists and it is published successfully. No configuration of two wheels can produce this motion.

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0$$

The robot can reach any pose, but not by moving *directly* toward it.

The Differential-Drive Robot



Constant	TurtleBot 3 Burger	Where it is written down
r wheel radius	0.033 m	stated explicitly in the URDF
L wheel separation	0.160 m	<i>not stated</i> — the distance between two joint origins
maximum wheel speed	14 rad/s	the motor datasheet

Two measurements of one physical robot. The whole model uses these two values.

Forward Kinematics: Wheels to Body

$$v = \frac{r(\omega_R + \omega_L)}{2}$$

$$\omega = \frac{r(\omega_R - \omega_L)}{L}$$

```
def forward(wL, wR):  
    v = r * (wR + wL) / 2  
    w = r * (wR - wL) / L  
    return v, w
```

Under the hood

The *sum* of the wheel speeds produces forward motion. The *difference* produces rotation. Understanding this removes the need to memorise the equations.

The Instantaneous Centre and the Turning Radius

```
$ ros2 run ee414_course simple_kinematics
wL= 10 wR= 10 -> v=+0.3300 m/s w=+0.0000 rad/s straight
wL= 5 wR= 10 -> v=+0.2475 m/s w=+1.0312 rad/s arc
wL= -10 wR= 10 -> v=+0.0000 m/s w=+4.1250 rad/s spin in place
wL= 0 wR= 10 -> v=+0.1650 m/s w=+2.0625 rad/s pivot
```

$$R = \frac{v}{\omega} \quad \text{distance to the Instantaneous Centre of Curvature}$$

Examine the last row

$R = 0.1650/2.0625 = 0.0800$ m, which is exactly $L/2$. When one wheel is stopped the robot rotates about *that wheel*, not about its own centre. The equations predict this before the robot demonstrates it.

Inverse Kinematics and Actuator Saturation

$$\omega_L = \frac{v - \omega L/2}{r} \qquad \omega_R = \frac{v + \omega L/2}{r}$$

```
def inverse(v, w):  
    wL = (v - w * L / 2) / r  
    wR = (v + w * L / 2) / r  
    return wL, wR
```

A robot does not reject an impossible command. It saturates.

$v = 0.5$, $\omega = 3.0$ requires 22.42 rad/s from the faster wheel. The Burger provides 14. It executes what it can and the path changes shape. This is the most common reason an open-loop square does not close. Scale BOTH v and ω by the same factor, or the requested curve changes.

From Velocity to Position: Integration

Given v and ω , determine x , y and θ

$$\dot{x} = v \cos \theta \quad \dot{y} = v \sin \theta \quad \dot{\theta} = \omega$$

	Euler	Exact arc
Assumption	orientation constant over Δt	the robot follows a circular arc
Error	halves as Δt halves; never zero	exact to 10^{-16} in every case
Use when	Δt is small, code must be simple	$\omega \neq 0$ and accuracy is required

Under the hood

The Euler method advances in the direction the robot faced at the *start* of the interval. During a rotation the robot does not hold that orientation for the whole interval, so the estimate falls short of the true arc at every step, in the same direction.

Wheel Odometry and Its Error Sources

Dead reckoning: accumulate every small step

Wheel odometry integrates the encoder readings. It has no external reference, so every error is **added to the total permanently**. The estimate never improves.

Source	What it does to the estimate
Wheel slip	distance recorded that the robot did not travel
Unequal tyre wear	a constant, undetected curve to one side
Incorrect r or L	every distance or rotation scaled by a fixed factor
Integration timestep	the error described on the previous slide

Week 7 adds a sensor that observes the environment. This is also why Nav2 exists.

Angle Normalisation

```
$ ros2 run ee414_course simple_angle_wrap
robot faces +176 deg      goal is at -176 deg      they are 8 deg apart
naive      target - facing = -352.0 deg
wrapped    normalize(t - f) = +8.0 deg
```

```
def normalize(angle):
    return math.atan2(math.sin(angle), math.cos(angle))
```

Always normalise the difference between two orientations

A controller given -352° rotates in the longer direction to make an 8° correction, every time the robot crosses the $\pm 180^\circ$ boundary.

Open-Loop Motion Control

```
seconds = DISTANCE / SPEED
msg.linear.x = SPEED
deadline = time.time() + seconds
while time.time() < deadline:
    pub.publish(msg)
    time.sleep(0.05)
stop(pub)
```

the entire control law

a topic does **not** latch: keep sending

an empty **Twist** **is** a full stop

What this code does not contain

No line reads the pose. The node stops because the timer has expired.

simple_move — commanded 2.000 m, measured 2.016 m.

The Open-Loop Square

```
for i in range(4):  
    hold(pub, SPEED, 0.0, SIDE / SPEED)    go straight  
    hold(pub, 0.0, TURN, math.radians(90) / TURN)    turn in place
```

```
start  x: 5.5444  y: 5.5444  theta: 0.000  
end    x: 5.6820  y: 5.4207  theta: 0.149    0.185 m out, 8.5 deg off
```

Two executions produce two different results

Nothing was measured, so nothing could be corrected. The errors of eight separate motions accumulated.

Closed-Loop Motion Control: Preview of Week 5

```
dx = GOAL_X - self.pose.x           it LOOKS
dy = GOAL_Y - self.pose.y
distance = math.hypot(dx, dy)
heading_error = normalize(math.atan2(dy, dx) - self.pose.theta)

msg.angular.z = clamp(K_ANGULAR * heading_error, MAX_W)
```

One idea separates this from every other program in this lecture

`error = goal - measured_position`. The other programs compute a duration in advance. This one reads the pose on every cycle and derives the velocity from the error.

Week 4 measures the error. Week 5 corrects it.

Laboratory Session

```
$ ros2 run turtlesim turtlesim_node           terminal 1, leave it running

$ ros2 run ee414_course simple_kinematics      the model, no simulator
$ ros2 run ee414_course simple_pose_reader    node, subscription, callback
$ ros2 run ee414_course simple_nonholonomic   sideways does nothing
$ ros2 run ee414_course simple_move           open loop, one division
$ ros2 run ee414_course simple_square         it does not close
$ ros2 run ee414_course simple_angle_wrap     the seam at 180 degrees
$ ros2 run ee414_course simple_go_to_goal     closed loop, it arrives
```

Before you begin

Run `conda deactivate` if your prompt shows an environment name. Between runs, use `ros2 service call /reset std_srvs/srv/Empty '{}'` to return the turtle to the centre.